Tree and Priority Queue

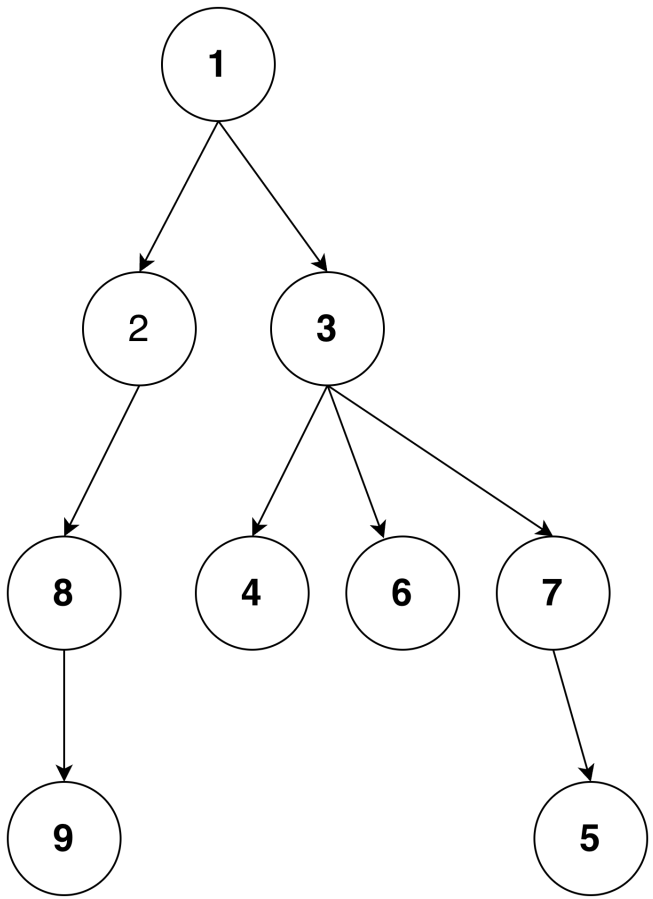
2026-1학기 부경대학교 알고리즘 스터디

목차

- 트리
  - 정의 및 용어
  - 트리 표현
  - 트리 순회
  - 이진 트리
- 우선순위 큐
  - 정의
  - 시간 복잡도
  - c++ 에서의 우선순위 큐

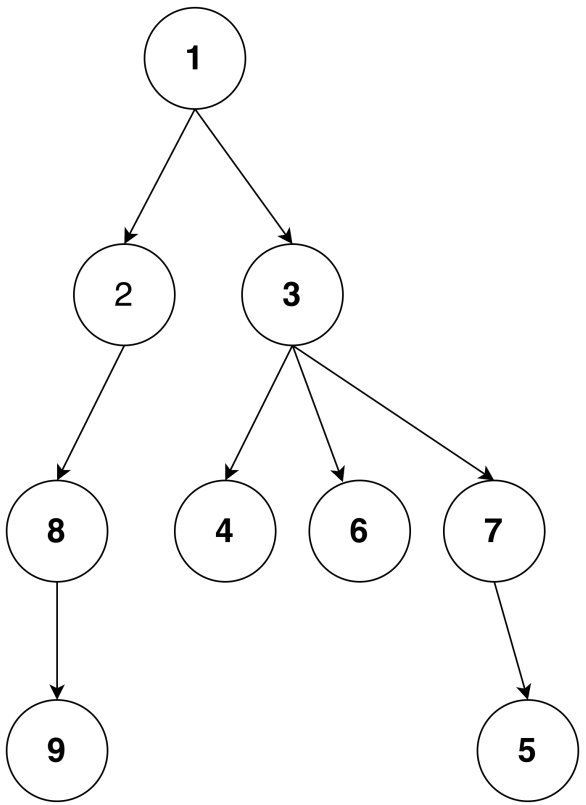
트리 - 정의

- 트리(Tree)란?
  - 트리는 사이클이 없으며 연결 되어있는 그래프를 의미함
  - 계층 구조를 표현하기 적합
  - 아래와 같은 특징을 가짐
    - ※ 1) 간선이 N – 1개 존재함
    - ※ 2) 두 정점을 잇는 경로는 하나만 존재함
  - DAG와 달리 하나의 부모 노드를 가짐



트리 - 용어

- 트리에서 사용되는 용어
  - 루트 : 최상위 노드
  - 리프 : 최하위 노드 (자식이 없는 노드)
  - 부모 노드 : 현재 노드의 상위 노드
  - 자식 노드 : 현재 노드의 하위 노드
  - 높이 : 루트에서 가장 먼 리프 노드까지의 거리
  - 깊이 : 루트에서 임의의 노드까지의 거리
  - 서브 트리 : 현재 노드를 루트로 하는 트리
  - 크기 : 트리를 구성하는 노드의 개수

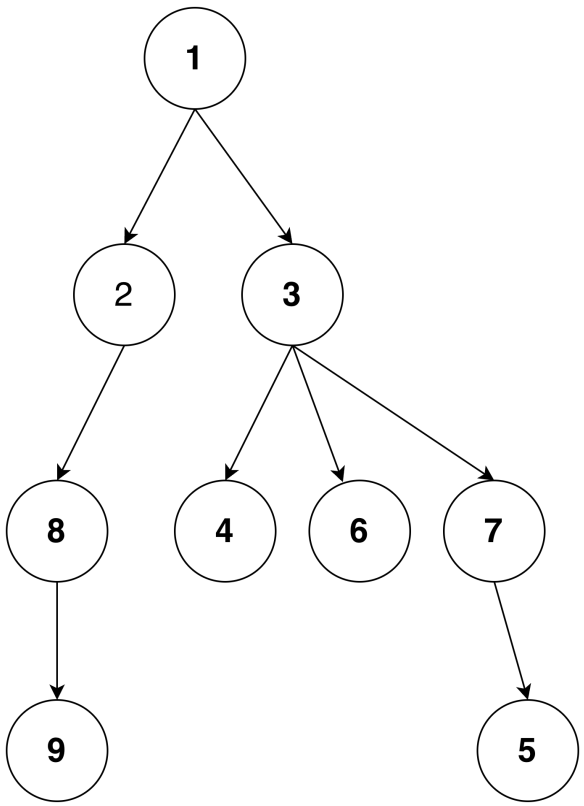


트리 - 표현

- 그래프처럼 인접 리스트로 표현 가능

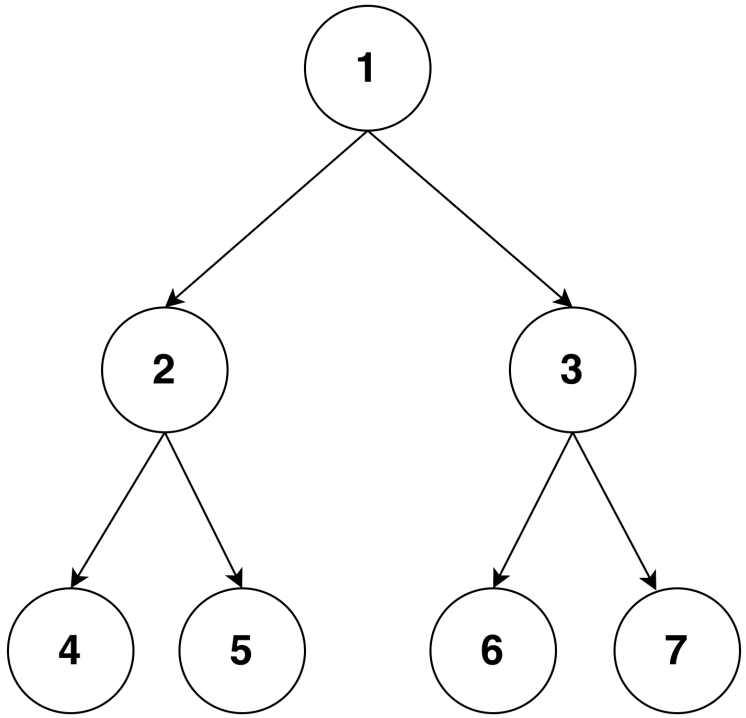
  ※ 부모 - 자식 관계를 그래프의 간선으로 표현 가능
  ※ 노드 : 부모 노드는 1 : 1관계이므로 배열로 표현 가능
  ※ 예시 트리를 아래와 같이 표현 가능하다

```
// child[i] : i 노드의 자식 리스트
vector<int> child[10];
// par[i] : i 노드의 부모 노드
int par[10];

child[1].push_back(2);
child[1].push_back(3);
par[2] = par[3] = 1;

child[2].push_back(8);
par[8] = 2;

child[3].push_back(4);
child[3].push_back(6);
child[3].push_back(7);
par[4] = par[6] = par[7] = 3;

child[7].push_back(5);
par[5] = 7;

child[8].push_back(9);
par[9] = 9;
```



트리 - 순회

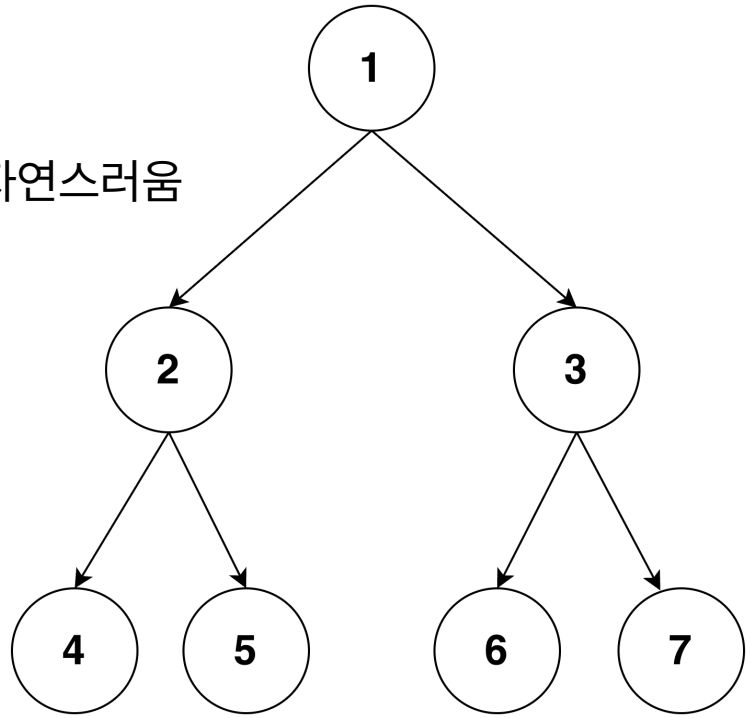
- 트리 순회 (Traversal)란?
  - 트리의 각 노드를 정확히 한 번씩만 방문하는 탐색하는 것
  - 전위 순회 (preorder) : 현재 노드 -> 왼쪽 노드 -> 오른쪽 노드 순으로 순회
    - ※ 1 -> 2 -> 4-> 5-> 3 -> 6 -> 7
  - 중위 순회 (inorder) : 왼쪽 노드 -> 현재 노드 -> 오른쪽 노드 순으로 순회
    - ※ 4 -> 2 -> 5 -> 1 -> 6 -> 3-> 7
  - 후위 순회 (postorder) : 왼쪽 노드 -> 오른쪽 노드 -> 현재 노드 순으로 순회
    - ※ 4 -> 5 -> 2 -> 6 -> 7-> 3 -> 1



트리 - 순회

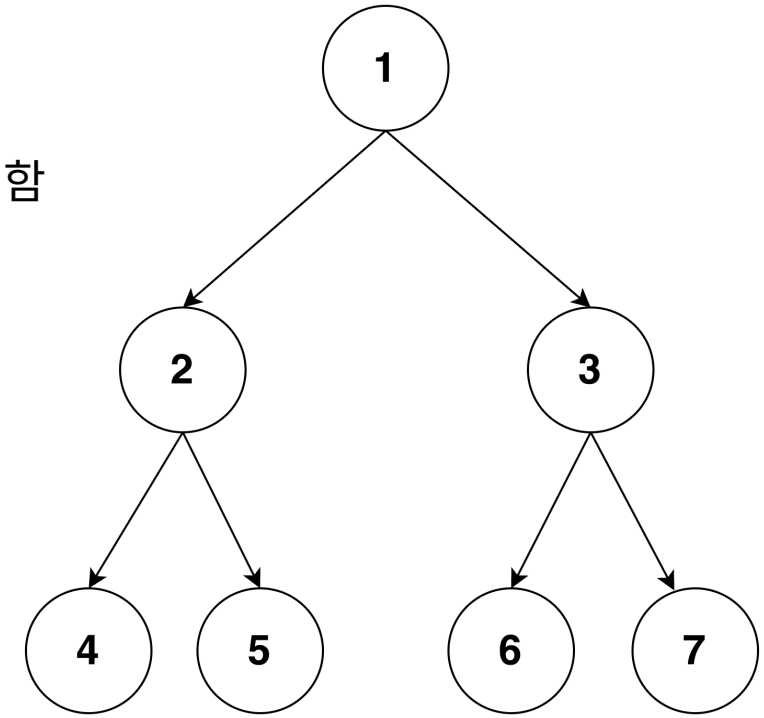
- 순회의 구현
  - 트리의 각 노드들 또한 하나의 서브 트리를 구성하므로, DFS로 재귀적 구현이 자연스러움
    - ※ 가령 예제 트리의 중위 순회는 아래와 같이 표현 가능하다

```
// child[x][0] : x노드의 왼쪽 자식
// child[x][1] : x노드의 오른쪽 자식
// 1번이 루트 노드, 자식이 없는 경우 0으로 설정
int child[8][2];

// 중위 순회 : 왼쪽->현재->오른쪽 순으로 탐색 (이진 트리(자식이 최대 2개)라고 가정)
void order(int node) {
    // 현재 노드 번호가 0번이라면 종료
    if(node == 0) return;
    // 순서대로 왼쪽 -> 현재 -> 오른쪽 순으로 출력!
    order(child[node][0]);
    cout << node << " ";
    order(child[node][1]);
}

int main() {
    child[1][0] = 2; child[1][1] = 3;
    child[2][0] = 4; child[2][1] = 5;
    child[3][0] = 6; child[3][1] = 7;

    order(1);
}
```



트리 - 순회

- 순회 활용
  - 재귀를 사용한 순회는 아래와 같이 부모 - 자식 노드들 간의 점화식 구현에 용이함
  - 현재 노드의 서브 트리 크기 = sum(자식 노드의 서브 트리 크기) + 1
    - ※ 1번 노드의 크기는 2번, 3번 노드의 서브 트리 크기의 합 + 1로 표현 가능
  - 현재 노드의 깊이 = 부모 노드의 깊이 + 1
  - 트리의 높이 = max(자식 노드의 서브 트리 높이) + 1



트리 순회 JUNGOL 9659

- 문제
  - 루트가 1이고, 노드의 개수가 N개인 트리에서, 아래 4가지 정보를 구하는 프로그램을 작성하자
  - 1) 루트와의 거리 2) 본인 포함 자손 노드의 개수 3) 가장 먼 자손과의 거리 4) 가장 먼 노드와의 거리
    - ※ 자손 노드란 현재 노드의 서브 트리에 속한 모든 노드(본인 제외)를 의미함.
- 풀이
  - 루트와의 거리 = 깊이, 본인 포함 자손 노드의 개수 = 서브 트리의 크기, 가장 먼 자손과의 거리 = 서브 트리의 높이
  - 따라서 순회 재귀 함수를 활용해서 구할 수 있다

트리 순회 JUNGOL 9659

- 문제
  - 루트가 1이고, 노드의 개수가 N개인 트리에서, 아래 4가지 정보를 구하는 프로그램을 작성하자
  - 1) 루트와의 거리 2) 본인 포함 자손 노드의 개수 3) 가장 먼 자손과의 거리 4) 가장 먼 노드와의 거리
    - ※ 자손 노드란 현재 노드의 서브 트리에 속한 모든 노드(본인 제외)를 의미함.
- 풀이
  - 가장 먼 노드와의 거리는 어떻게 구할까?
  - 만약 본인의 서브 트리 내부에 존재한다면, down이 가장 먼 노드와의 거리가 될 것
  - 그렇지 않다면 부모 노드를 타고 올라가면서, 형제 노드의 down 값과 비교

```cpp
int N;
// 순서대로 부모 노드, 노드의 깊이, 서브 트리의 높이, 서브 트리의 크기를 의미
int par[SIZE], dep[SIZE], height[SIZE], sz[SIZE];
// 노드의 자식 리스트
vector<int> child[SIZE];

// cur을 현재 노드, prev를 이전 노드(cur의 부모 노드) 라고 하자
void order(int cur, int prev) {
    // 현재 노드의 높이는 부모 노드의 높이 + 1
    dep[cur] = dep[prev] + 1;

    // 서브 트리에 자기 자신도 포함되므로 + 1
    sz[cur] += 1;

    // 만약 자식이 없다면 더 탐색하지 않고 종료
    if(child[cur].empty()) return;

    for(int next : child[cur]) {
        order(next, cur);        // 순회
        sz[cur] += sz[next];     // 서브 트리의 크기 += 자식 노드의 서브 트리 크기

        // 높이 = max(자식 노드의 높이 + 1)
        height[cur] = max(height[cur], height[next] + 1);
    }
}
```

```cpp
int main() {
    cin.tie(0)->sync_with_stdio(false);
    cin >> N;

    for(int i = 0; i < N - 1; i++) {
        int cid, pid;
        cin >> cid >> pid;

        child[pid].push_back(cid);
        par[cid] = pid;
    }

    order(1, 0);
```

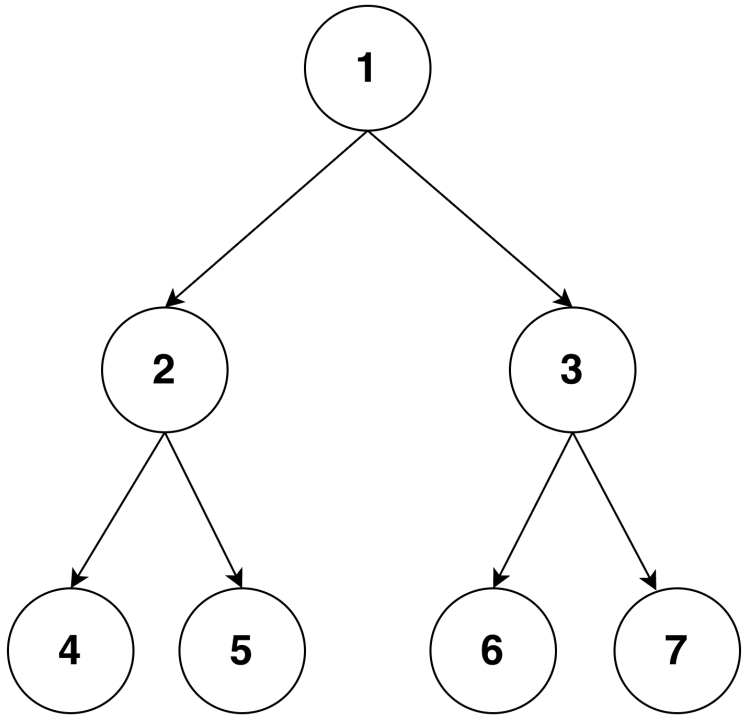
- 순회 함수를 수정해서 트리의 높이, 깊이, 크기를 모두 구함

```
// 높이, 깊이, 크기를 구할 노드 번호 입력
int node;
cin >> node;

// 해당 노드에서 가장 먼 거리의 노드 탐색
int cur = node, longestDist = 0;

// 부모 노드가 존재할때까지 부모 노드를 타고 올라감
while(par[cur] ≠ 0) {
    int prev = par[cur];

    // 1) 현재 탐색중인 노드와의 거리가 최대일 수 있음
    longestDist = max(longestDist, dep[node] - dep[prev]);

    // 2) 그렇지 않다면, 형제 노드의 높이 -> 부모 노드 -> cur 노드 -> node간의 거리가 최대일 수 있음
    for(int sibling : child[prev]) {
        if(sibling == cur)
            continue;
        longestDist = max(longestDist, height[sibling] + 2 + dep[node] - dep[cur]);
    }

    cur = par[cur];
}

// 가장 먼 거리의 노드는 height, longesDist 중 최대값
longestDist = max(longestDist, height[node]);
```

- 가장 먼 거리의 노드는 현재 서브 트리의 높이 혹은 루트까지의 경로를 타고 올라가며 확인할 수 있음

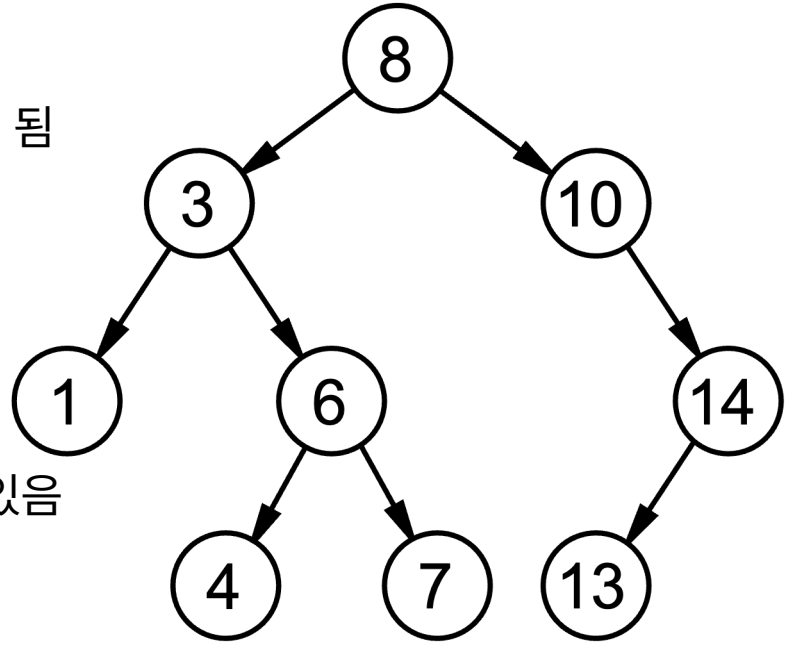
트리 – 이진 트리

- 이진 트리 (binary tree)란?
  - 각 노드의 자식의 노드가 최대 2개인 트리
    - ※ 마지막 레벨을 제외하고 모든 레벨이 채워져있고, 마지막 레벨은 왼쪽부터 채워진 트리를 완전 이진 트리라고 정의
  - 다양한 자료구조의 표현의 기반이 됨
  - 1) 이진 탐색 트리
    - ※ 모든 노드가 왼쪽 자식을 현재 노드보다 작은 값, 오른쪽 자식을 현재 노드보다 큰 값이 되도록 만든 트리
    - ※ 자료구조 시간에 배운 set, map이 균형 잡힌 이진 탐색 트리로 구현되어 있음
  - 2) 세그먼트 트리
    - ※ 배열의 구간 정보를 포함하는 트리
    - ※ 현재 노드가 left ~ right 구간의 정보를 다룬다고 할 때, 왼쪽 노드와 오른쪽 노드가 구간의 절반씩을 다루는 트리
    - ※ 구간 합 등 구간 질의를 O(log N)에 처리
  - 3) 최소/최대 힙
    - ※ 노드가 두 자식 노드 보다 작거나/크도록 하는 완전 이진 탐색 트리를 최소/최대 힙으로 정의
    - ※ 우선순위 큐는 힙으로 구현 가능



트리 – 이진 탐색 트리

- 이진 탐색 트리(BST, Binary Search Tree)란?
  - 각 노드가 하나의 key를 가지고, “왼쪽 서브 트리의 모든 key < 현재 key < 오른쪽 서브 트리의 모든 key” 를 만족하는 이진 트리
    - ※ 각 노드가 최대 2개의 자식 노드를 갖는 트리를 이진 트리라고 함
  - 값을 조회할 때 현재 노드의 값에 따라 왼쪽 혹은 오른쪽으로 내려가며 탐색하면 됨
    - ※ 가령, 오른쪽 트리에서 4를 찾는다면 8 -> 3 -> 6 -> 4로 이동



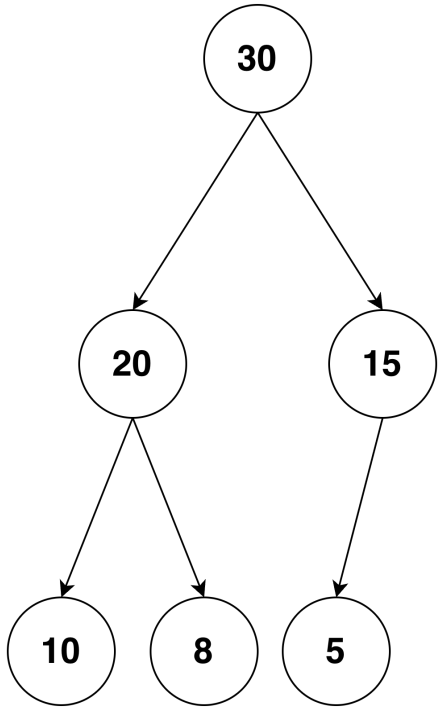
- 이진 탐색 트리의 문제점
  - Key의 삽입 순서에 따라서 트리의 모양이 달라짐 -> 한쪽으로 길게 늘어질 수 있음
    - ※ 이러한 모양의 이진 탐색 트리는 검색이 비효율적
  - 이러한 트리는 높이가 높아 탐색, 삽입, 삭제가 비효율적

트리 – 이진 탐색 트리

- 균형 잡힌 이진 탐색 트리 (Balanced BST)란?
  - 높이가 **O(log N)**이 되도록 유지되는 이진 탐색 트리
  - AVL 트리, Red-Black 트리 등이 존재
- BST의 시간복잡도
  - 이진 트리에서의 삽입, 삭제, 검색의 최악의 경우는 O(트리의 높이)
  - 따라서 삽입 삭제 검색 모두 O(log N)의 시간 복잡도를 가짐
    - ※ 높이를 log N 수준으로 유지하는 회전 알고리즘 또한 O(log N)의 시간 복잡도를 가짐

우선순위 큐 – 정의

- 우선순위 큐 (priority queue)란?
  - 우선 순위가 높은 데이터가 먼저 나오는 추상체
    - ※ 일반 queue는 선입선출
  - 우선 순위는 큰 값, 작은 값 등이 될 수 있음
  - 일반적으로 힙으로 구현
    - ※ 완전 이진 트리 구조로, 노드의 우선순위가 자식의 우선 순위보다 높도록 유지하는 자료구조
    - ※ 예제는 최대힙으로, 전 구간에서 부모 노드 >= 자식 노드를 만족



우선순위 큐 – 시간 복잡도

- 연산
  - push : O(log N)
    - ※ 내부 힙을 재정렬하는데 약 O(log N)의 비용이 발생
  - pop : O(log N)
    - ※ push와 마찬가지로 내부 힙을 재정렬하는데 약 O(log N)의 비용이 발생

- 우선순위 큐를 사용하는 이유?

	우선순위 높은 값 조회	값 삽입
일반 배열	O(N)	O(1)
정렬된 배열	O(1)	O(N)
우선순위 큐	O(1)	O(log N)

  - 선형 자료구조는 조회와 삽입에 트레이드 오프가 존재

우선순위 큐 – c++ 에서의 우선순위 큐

- queue 헤더에 priorit_queue를 주로 사용

- priority_queue<자료형, 컨테이너, 비교자> 형태로 선언 가능
  - ※ 자료형 : 큐에 들어갈 자료형
  - ※ 컨테이너 : 원소를 저장할 컨테이너 (디폴트는 vector)
  - ※ 비교자 : 우선 순위를 매기는 비교자 (디폴트는 less, 즉 큰 값이 top에 위치함)

```
int main() {
    // 우선순위 큐 pq 선언
    // 컨테이너, 비교자는 선언하지 않을 시 디폴트 값
    priority_queue<int> pq;
    pq.push(1);
    pq.push(11);
    pq.push(7);

    while(!pq.empty()) {
        cout << pq.top() << " ";
        pq.pop();
    }
    // output:
    // 11 7 1
}
```

우선순위 큐 – c++ 에서의 우선순위 큐

- 비교자 정의 방법
  - 1) 기본 비교자 사용

```
priority_queue<int, vector<int>, less<int> > pq1;
priority_queue<int, vector<int>, greater<int> > pq2;
```

  - 2) 구조체 비교자 사용

※ sort 함수의 비교 연산자와 같이 약한 오더링을 따라야 함

```
// 비교자 구조체 정의
struct mod3 {
    // 3으로 나눈 나머지가 작은 값이 우선 순위가 높도록 비교자 정의
    bool operator()(int a, int b) {
        return a % 3 > b % 3;
    }
};
```

우선순위 큐가 3개 JUNGOL 4892

- 문제
  - 큰 값, 작은 값, 절대값이 각각 우선순위인 정수 타입 우선순위 큐 3개가 있다.
  - 총 q개의 쿼리가 주어진다. 각 쿼리가 0이 아니라면 각 큐에 push, 0이라면 pop후 pop을 출력한다.
- 풀이
  - 큰 값 우선 순위 -> less, 작은 값 우선 순위 ->greater 비교자를 쓰면 됨
  - 절대값은 구조체 선언을 사용해 비교자를 생성해보자

```
// 구조체 비교자 생성
struct compare {
    // 절대값이 작은게 우선순위가 되도록 비교 연산 수행
    bool operator()(int a, int b) {
        if(abs(a) == abs(b))
            return a > b;
        return abs(a) > abs(b);
    }
};
```

- 위와 같이 비교자 구조체를 생성해서 절대값 pq를 구현해보자

```cpp
// 구조체 비교자 생성
struct compare {
    // 절대값이 작은게 우선순위가 되도록 비교 연산 수행
    bool operator()(int a, int b) {
        if(abs(a) == abs(b))
            return a > b;
        return abs(a) > abs(b);
    }
};

int main() {
    cin.tie(0)->sync_with_stdio(false);

    priority_queue<int, vector<int>, less<int> > max_pq;
    priority_queue<int, vector<int>, greater<int> > min_pq;
    priority_queue<int, vector<int>, compare > abs_pq;

    int q;
    cin >> q;
    // 쿼리 입력받기
    while(q--) {
        int x;
        cin >> x;
        // x가 0이 아니라면 push
        if(x != 0) {
            max_pq.push(x); min_pq.push(x); abs_pq.push(x);
        }
        // 0이라면 pop하기
        else {
            // 값이 없다면 -1 한번만 출력
            if(max_pq.empty())
                cout << -1 << endl;
            // 값이 있다면 top 출력 후 pop
            else {
                cout << max_pq.top() << " "
                     << min_pq.top() << " "
                     << abs_pq.top() << endl;

                max_pq.pop(); min_pq.pop(); abs_pq.pop();
            }
        }
    }
}
```